REPORT · 2026 · 19

对战盒子 · 代码逻辑说明书

LOL 开黑抽签与积分排行系统 · 全量逻辑技术文档

文档说明

本文档是「对战盒子」（原名「5v5 均衡抽签」）的完整代码逻辑说明书，覆盖系统架构、数据模型、计分规则、全部后端接口、抽签算法、投票重抽、LOL 战绩自动导入、智能分析算法、前端渲染逻辑、流量优化与运维要点。文档内容基于项目当前最新代码（app.py 共 1031 行、index.html 共 1704 行）逐行核对整理，可直接作为二次开发、交接与故障排查的依据。

项目项	内容
项目名称	对战盒子（LOL 开黑抽签与积分排行）
业务定位	微信群内多人共享的 5v5/4v4/3v3 开黑分组与积分排行工具
后端	Python + Flask（app.py，1031 行）
前端	原生 HTML + CSS + JavaScript 单文件单页应用（index.html，1704 行，无外部依赖）
数据库	SQLite（elo_draw.db），WAL 模式默认，6 张业务表
数据同步	LOL 官方客户端 LCU 接口 → PowerShell 脚本 → HTTP 上报
内网地址	http://127.0.0.1:9529
公网地址	https://124777lk2jf53.vicp.fun（花生壳固定域名，映射 127.0.0.1:9529）
运行端口	9529（Flask，host=0.0.0.0）

一、系统架构总览

系统采用「单机 Flask 服务 + 内网穿透 + 多端浏览器」的轻量架构。Flask 服务运行在群主自己的 Windows 电脑上，数据全部保存在本机 SQLite 文件里，不依赖任何云服务；公网访问通过花生壳固定域名映射到本机 9529 端口实现。微信群成员通过同一个公网地址访问同一份数据，做到「所有人看到同一份抽签分组与积分」。

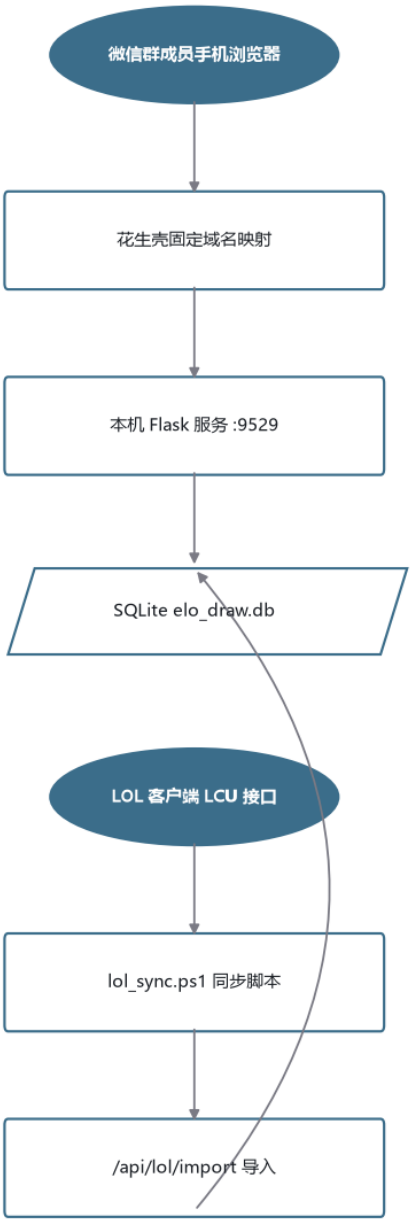


Figure 1: 对战盒子整体数据流

数据流有两条主线：其一是「抽签线」——用户在前端勾选玩家并抽签，分组结果提交到服务器共享展示，并在 5 分钟后自动失效；其二是「战绩线」——数据源电脑运行 lol_sync.ps1 读取 LOL 客户端自定义对局，推送到 /api/lol/import 接口，服务器自动建局、计分并重算智能分析。

二、运行环境与部署

2.1 服务端口与进程

Flask 服务固定监听 9529 端口，启动方式为 app.run(host='0.0.0.0', port=PORT, debug=False)，即局域网内其他设备也可直接通过 http://<本机IP>:9529 访问。启动时依次执行三步初始化：init_db()（建表与历史迁移）、backfill_analytics()（补档历史智能分析）、再启动一个守护线程 _analytics_scheduler（每晚 03:00 自动定版前一晚的分析）。

2.2 一键启动脚本

文件	作用
启动抽签.bat	入口，调用 wscript 执行「启动抽签.vbs」
启动抽签.vbs	核心启动器：探活 Flask、未运行则拉起 service_start.bat、取公网地址、打开浏览器并弹窗显示分享链接
service_start.bat	先检查并自动 pip 安装 flask，再执行 python app.py 常驻运行（窗口需保持打开）
lol_sync.bat	LOL 同步工具入口，自动申请管理员权限并执行 lol_sync.ps1
lol_sync.ps1	读取 LCU 令牌→拉取自定义对局→预览或上报服务器

「启动抽签.vbs」的执行逻辑：① 若 http://127.0.0.1:9529/api/state 探活失败，则静默运行 service_start.bat 并轮询最多 40 秒等待就绪；② 调用 /api/public-url 获取公网地址，若返回空则拉起 cpolar 临时隧道作备用；③ 用系统默认浏览器打开本机页面，并弹窗显示「群分享链接」同时复制到剪贴板。该脚本支持诊断模式：cscript //nologo 启动抽签.vbs test 只输出诊断信息、不弹窗、不开浏览器。

2.3 PWA 应用化（添加到手机桌面）

前端通过 manifest.webmanifest 与三类图标实现「添加到桌面」后以独立窗口打开、像原生 App 一样使用。服务器为 /manifest.webmanifest、/icon-192.png、/icon-512.png、/apple-touch-icon.png 提供专门路由并设置 7 天浏览器缓存。

manifest 字段	取值
name / short_name	对战盒子
description	开黑抽签与积分排行

start_url / scope	/
display	standalone（独立窗口，隐藏浏览器地址栏）
orientation	portrait（竖屏）
background_color	#f0f6ff
theme_color	#2563eb
icons	icon-192.png（any）、icon-512.png（any maskable）

三、数据模型

数据库为单文件 SQLite（elo_draw.db），共 6 张表。init_db() 在每次启动时以 CREATE TABLE IF NOT EXISTS 幂等建表，并对旧库执行 ALTER TABLE 补列，保证老数据平滑升级。

3.1 players（玩家与积分池）

字段	类型	说明
id	TEXT PK	玩家唯一 id，初始玩家为 p_init_N，新增为 p<毫秒时间戳>
name	TEXT UNIQUE	昵称，唯一约束（重名不可添加）
score3	INTEGER	3v3 榜积分，默认 0
score4	INTEGER	4v4 榜积分，默认 0
score5	INTEGER	5v5 榜积分，默认 0
delta	INTEGER	最近一次积分变动量（正/负），用于前端展示涨跌
score	INTEGER	历史遗留列（旧单一积分），迁移时已把值并入 score5

3.2 games（对局记录）

字段	类型	说明
id	INTEGER PK	自增主键
time	TEXT	对局时间，格式 MM-DD HH:MM（用于比赛日归组）
team_a	TEXT	A 队成员 JSON 数组，元素含 id/name/score，LOL 局还含 kills/deaths/assists/dmg/gold/tk/heal/champ
team_b	TEXT	B 队成员 JSON 数组，结构同上
result	TEXT	胜负，'A' / 'B' / NULL（未结算）
draw_type	TEXT	分组方式：balanced / random / lol（LOL 自动导入）

kill_ids	TEXT	四杀标记 JSON 数组，id 重复出现 = 同一人多次标记
penta_ids	TEXT	五杀标记 JSON 数组（仅 5v5），同上
lol_game_id	INTEGER	LOL 对局唯一编号，用于导入幂等去重，手动局为 NULL

3.3 analytics（智能分析存档）

字段	类型	说明
day	TEXT PK	比赛日，格式 MM-DD
games_count	INTEGER	该比赛日已结算对局数
content	TEXT	四榜单结果 JSON 数组（bestPair/best/worst/worstPair）
generated_at	TEXT	生成时间 MM-DD HH:MM

3.4 current_draw（当前共享抽签分组）

字段	类型	说明
id	INTEGER PK	自增，只保留最新一条
draw_type	TEXT	balanced / random
mode	INTEGER	每队人数（3/4/5）
team_a / team_b	TEXT	两队成员 JSON
votes	TEXT	重抽投票者 id 列表 JSON
created_at	REAL	创建时间戳（秒），用于 5 分钟 TTL 判定

3.5 last_draw（最近一次抽签名单）

字段	类型	说明
id	INTEGER PK CHECK(id=1)	固定单行
player_ids	TEXT	上次抽签参赛玩家 id 列表 JSON（持久，全群共享）

updated_at	REAL	更新时间戳

3.6 meta（迁移标记）

key	含义
rescale_5x_v1	旧规则 ±1 换新规则 ±5，历史积分×5，仅执行一次
penta_tier_v1	五杀加档迁移标记（5v5 旧 kill_ids 语义为五杀，迁入 penta_ids 并补足差额）

3.7 初始化与历史迁移

init_db() 中的关键一次性操作，均以 meta 表标记保证幂等，只执行一次：

1. 补列迁移：为 players 补 score3/score4/score5（若存在旧列 score 则其值迁入 score5）；为 games 补 draw_type/kill_ids/penta_ids/lol_game_id。
2. 积分 ×5 换算（rescale_5x_v1）：旧规则胜 +1 / 负 −1 改为 ±5，历史积分整体乘以 5。
3. 五杀加档迁移（penta_tier_v1）：旧数据中 5v5 的 kill_ids 语义是「五杀标记」（当时按 ±1 计），迁入新的 penta_ids 列，并按新规则为胜方每人补 +（KILL_BONUS_PENTA − KILL_BONUS_QUADRA）× 标记数、败方每人补 −同值；4v4 的 kill_ids 保持四杀 ±1 不变。
4. 初始玩家注入：仅当 players 表为空时，写入 11 名初始玩家及其 5v5 初始积分。

> **初始玩家池（含 5v5 初始分）：UZI 5、神撞大队长 3、方特永远的神QvQ 1、The 棍 1、卢本伟 0、米兰的小铁锤 −1、秒退道士 −1、faker −1、OttO −1、抑郁快滚开 −3、ZEUS −3。**

四、计分与标记规则（核心业务）

4.1 积分常量

PYTHON

```
WIN_SCORE = 5          # 胜方每人 +5
LOSE_SCORE = 5         # 败方每人 -5
KILL_BONUS_QUADRA = 1   # 四杀加成（每次）
KILL_BONUS_PENTA = 2    # 五杀加成（每次，仅 5v5）
```

4.2 结果录入算法（/api/games//result）

该接口支持「首次录入」与「修改结果」两种用途，先回退旧结果、再应用新结果，保证幂等。对应模式由每队人数决定积分列（3→score3、4→score4、5→score5）。

1. 校验 result 必须为 A 或 B；读取对局，计算该场模式积分列 col 与人数 n。
2. 回退旧结果：若原 result 已存在，胜方每人 −5、败方每人 +5；再回退旧加成（旧四杀次数×1 + 旧五杀次数×2，仅统计胜方成员的标记）。
3. 应用新结果：胜方每人 +5 + 加成，败方每人 −5 − 加成；同时写入 players.delta 记录本次变动量。
4. 加成计算：只保留「属于胜方成员」的标记；存库时同一 id 重复出现 N 次表示标记 N 次，加成 = 四杀次数×1 + 五杀次数×2，全队共享同一加成值。
5. 写入 games.result / kill_ids / penta_ids，提交后调用 regen_day() 重算该比赛日智能分析。

4.3 四杀与五杀标记规则

模式	四杀	五杀
3v3	不可标记	不可标记
4v4	可标记，胜方全员 +1 / 败方全员 −1	不可标记
5v5	可标记，胜方全员 +1 / 败方全员 −1	可标记，胜方全员 +2 / 败方全员 −2

关键约束：同一人可在同一局内多次标记（可叠加）；标记只对胜方成员生效，非胜方成员的标记会被过滤丢弃；界面上五杀输入组带有提示「五杀已含四杀，勿再勾下方四杀」，避免重复计分。

4.4 撤销对局（/api/games//undo）

若该局已结算，先按该场模式回退胜方 +5、败方 −5，再分级回退四杀（±1）/五杀（±2）全队加成，最后删除该局记录并重算该比赛日分析（该日已无对局则删除分析存档）。

4.5 积分清零（/api/clear-scores）

将全部玩家的 score3 / score4 / score5 / delta 置 0，不做删除玩家操作。

五、比赛日规则

「比赛日」是智能分析与战绩页筛选的时间单位。规则为：18:00 ~ 23:59 归当天（今晚）；次日 00:00 ~ 06:00 以及白天 06:00 ~ 18:00 均归前一晚。该规则由后端的 game_day_key() 与前端的 gameDayKey() 双端同步实现。

PYTHON

```
def game_day_key(t):
    # t 形如 'MM-DD HH:MM'
    h = int(小时)
    if h >= 18: return '当天MM-DD'
    return prev_day_str('当天MM-DD')  # 凌晨与白天归前一晚
```

配套逻辑：① 每晚凌晨 03:00，_analytics_scheduler 线程自动重算前一晚（即当前比赛日）的分析存档，实现「定版」；该线程每 600 秒检查一次，服务重启不受影响、到点自动补跑。② 成绩页默认视图：18 点后默认看今晚，18 点前默认看昨晚；若该日无记录则回退到最近有记录的一天。③ 前端 prevDayStr / currentDayKey 与后端算法保持一致，避免前后端显示不一致。

六、后端 API 接口详解

6.0 接口总览

方法	路径	作用
GET	/	返回 index.html 首页
GET	/manifest.webmanifest	PWA 清单
GET	/icon-.png	PWA 图标（仅 192/512）
GET	/apple-touch-icon.png	iOS 图标
GET	/api/state	全量状态：玩家、对局、当前抽签、上次名单
POST	/api/draw/submit	提交抽签分组，存为共享当前分组
POST	/api/draw/vote	投重抽票，达半数解散分组
DELETE	/api/draw	手动删除当前抽签分组

POST	/api/players	新增玩家
DELETE	/api/players/	删除玩家
POST	/api/games	创建对局（未结算）
POST	/api/games//result	录入/修改对局结果并计分
POST	/api/games//undo	撤销对局并回退积分
GET	/api/analytics	读取已存档的智能分析
POST	/api/lol/import	LOL 战绩批量导入
GET	/api/public-url	对外分享地址（固定域名优先）
POST	/api/clear-scores	积分全部清零
POST	/api/import	批量导入玩家（仅玩家池为空时）

6.1 GET /api/state

返回 { players, games, draw, last_pick }。players 为全部玩家字段；games 按 id 升序，逐条把 team_a/team_b/kill_ids/penta_ids 反序列化为对象；draw 为当前共享分组（若存在且未过期，含 draw_type/mode/teamA/teamB/votes_count/threshold/age，过期则删除并返回 null）；last_pick 为上次抽签参赛玩家 id 列表。

6.2 POST /api/draw/submit

项	说明
入参	drawType（balanced/random，非法回退 balanced）、mode、teamA[]、teamB[]
校验	两队人数相同且 3–5 人；所有队员 id 必须在玩家池中，否则返回失败并指出具体昵称
逻辑	删除旧的 current_draw，插入新分组（票数清零、created_at=当前时间），并用 INSERT OR REPLACE 更新 last_draw 的参赛名单
出参	{ ok: true } 或 { ok:false, msg }

6.3 POST /api/draw/vote

入参 voter（每台设备一个随机投票身份）。取当前分组，若已超过 5 分钟 TTL 则删除并提示过期；阈值 threshold = 每队人数（10 人场为 5 票、8 人场为 4 票、6 人场为 3 票）。同一 voter 重复投票不计数。当票数达到阈值时，删除 current_draw（即解散分组）并返回 met=true；未达标则更新票数并返回 met=false。前端在 met=true 时刷新并提示「半数以上同意，分组已解散，请重新选人抽签」。

6.4 POST /api/players 与 DELETE /api/players/

新增玩家：昵称去空格后非空校验、唯一性校验（重复返回「该昵称已存在」），新 id 为 p + 毫秒时间戳，三种模式积分与 delta 均初始为 0。删除玩家：直接按 id 删除（前端会二次确认，提示积分记录一并清除）。

6.5 POST /api/games

创建一局未结算对局。校验每队 3–5 人且两队人数相同；draw_type 仅接受 balanced/random，非法回退 balanced；time 取服务器本地时间 MM-DD HH:MM；返回新对局 id。说明：当前版本抽签页已改为「抽签结果不入库、胜负由数据源同步」，该接口保留用于兼容与手动流程。

6.6 POST /api/games//result 与 /undo

入参 result（A/B）、killIds[]、pentaIds[]（数组内 id 重复出现 = 多次标记）。详细算法见「四、计分与标记规则」。撤销接口见 4.4 节。

6.7 GET /api/analytics

按比赛日倒序返回全部存档：{ ok:true, days:[{ day, games, rows, generatedAt }] }。缺档的比赛日由写入路径（录入/导入/撤销后 regen_day）或凌晨 03:00 调度自动补齐。

6.8 GET /api/public-url

对外分享地址获取：优先探测花生壳固定域名 https://124777lk2jf53.vicp.fun（HEAD 请求探活，4 秒超时），通则返回；不通则依次尝试 cpolar 本地管理页（127.0.0.1:9203/9201/4040 的 http/in 路径），用正则从隧道列表中挑选 LocalAddr 指向 9529 的公网地址（https 优先）。全部失败返回 { ok:false, url:null }。

6.9 POST /api/import

批量导入玩家：仅当玩家池为空时可用，否则返回「玩家池非空，无法导入」；逐条写入 name 与 score（写入 score5）。

七、抽签算法

系统支持两种分组算法，前端与后端各实现一份且规则完全一致。当前版本界面上「均衡抽签」入口已隐藏，drawMode 强制为 random，实际使用随机抽签。

7.1 随机抽签 randomDraw

对选中玩家整体洗牌（Fisher–Yates），前一半为 A 队、后一半为 B 队。逻辑直观、实现简单，是当前默认方式。

7.2 均衡抽签 balancedDraw

按积分降序排序后两两配对，每轮由「当前总分较低的队」先拿高顺位选手，从而削峰填谷、避免强强同队。最后对两队内部再洗牌打乱顺序。

JAVASCRIPT

```
function balancedDraw(list){
  var sorted = list.sort(function(a,b){ return b.score - a.score; });
  var A=[], B=[], sumA=0, sumB=0;
  for(var i=0; i<sorted.length; i+=2){
    var first = (sumA<=sumB) ? A : B;
    var second = (first===A) ? B : A;
    if(sorted[i]){ first.push(sorted[i]); (first===A?sumA:sumB)+=sorted[i].score; }
    if(sorted[i+1]){ second.push(sorted[i+1]); (second===A?sumA:sumB)+=sorted[i+1].score; }
  }
  return { A: shuffle(A), B: shuffle(B) };
}
```

7.3 抽签人数约束

抽签仅支持 6 / 8 / 10 人（对应 3v3 / 4v4 / 5v5），「开始抽签」按钮仅在此三种人数下可用；每队人数 = 总人数 / 2，积分榜相应归入 score3 / score4 / score5。前端在抽签前会先请求 /api/state 预检服务器是否已有存活分组，有则拦截本次抽签并同步现有分组（多人并发保护）。

7.4 分组有效期

抽签结果仅作赛前分组展示，不入库、不涉及胜负，服务器端 TTL = 5 分钟（DRAW_TTL = 300 秒）。前端有倒计时显示，到期自动收起并恢复「开始抽签」；服务器在读取状态时若发现过期即删除。

八、投票重抽流程

分组不满意时，任一端可双击分组卡片或点「投票重抽」投出一票（每台设备一票，身份存 localStorage 的 elo_voter_id）。票数达「每队人数」阈值（即半数）后由服务器解散分组，所有人回到选人界面重新勾选抽签。该设计取代了早期「服务器直接重抽」的做法，把决定权交还给全员。

1. 前端 voteRedraw() 调 /api/draw/vote，携带 voter 身份。
2. 服务器累加票数；未达标返回 { votes, threshold, met:false }，前端更新「□ 重抽 n/threshold」徽标。
3. 达标后服务器删除 current_draw 并返回 met:true，任一端刷新即可看到分组消失、选人区恢复。

> **投票是「解散分组」而不是「立刻重抽」：达标后全员回到选人界面，由大家重新确认参赛名单再抽签，避免在同一批人身上反复重抽而失去公平性。**

九、LOL 战绩自动导入

9.1 数据源与采集

lol_sync.ps1 运行在安装了 LOL 客户端的电脑上（需管理员权限读取 LCU 令牌）。它从 LeagueClientUx.exe 命令行解析 --app-port 与 --remoting-auth-token，用 Basic 认证访问本地 LCU 的国服端点，拉取最近 N 天（默认 6 天）的自定义对局列表并自动翻页，再逐局拉取详情。同步工具提供两种模式：[1] 预览（核对名单/胜负/多杀，不入账）；[2] 同步（上报服务器自动计分）。

9.2 上报与幂等去重（/api/lol/import）

脚本把每局整理为 { gameId, gameCreation, participants[] } 结构 POST 到服务器。服务器按 gameId 幂等去重：同一 LOL 对局只导入一次；若该局此前已导入但当时缺少 KDA/伤害/经济/承伤/英雄等明细，而本次数据带上了，则只补全这些字段（积分与胜负不动），返回 status=updated。

9.3 建局与计分

1. 拆分两队：标准 teamId 100（蓝方）→ A 队、200（红方）→ B 队；非标准键按出现顺序。
2. 校验两队人数相同且 3–5 人；逐一匹配玩家池，存在未入池的名字则该局跳过并回报原因。
3. 判定胜负：恰好一队全员胜；分队后据此得到 result（A/B）。
4. 计分：与手动录入同规则（胜 +5 + 加成 / 负 −5 − 加成；四杀×1、五杀×2），按人数归入对应模式榜。
5. 写库：games 记录 draw_type='lol'、lol_game_id、time 取对局创建时间（MM-DD HH:MM），成员快照含 KDA/伤害/经济/承伤/治疗/英雄名。
6. 导入完成后对受影响的比赛日统一 regen_day() 重算智能分析。

9.4 多杀净化（防重复计分）

脚本在采集阶段即做净化：五杀过程中必然伴生一次四杀播报，quadraKills 字段可能把五杀那次也计入。因此按最高档结算——净四杀 = 四杀字段 − 五杀次数（不为负），保证一次五杀绝不重复计成「四杀 + 五杀」。

POWERSHELL

```
# 净四杀 = 四杀字段 - 五杀次数（不为负）
quadra = if ($quadraRaw -gt $penta) { $quadraRaw - $penta } else { 0 }
```

9.5 服务器地址配置

lol_sync.ps1 第 22 行 $ServerUrl 固定为花生壳域名 https://124777lk2jf53.vicp.fun。由于花生壳映射已一次性买断至 2099 年，该地址永久不变，无需再随隧道变更而修改。脚本依赖本机花生壳服务在线；电脑关机时群页面暂时打不开，开机自动恢复。

十、名字归一化

LOL 的 Riot ID 可能使用繁体字（如「錘/锤」），与玩家池中的简体昵称不匹配，因此导入时做名字归一化容错。normalize_name()
去除首尾空格、转小写，再用内置的繁简对照表（TS_MAP，约 130 组常用繁简字）逐字归简。匹配顺序为：先精确匹配（含大小写），再归一化匹配。玩家池在导入开始时预建「归一化名 → 玩家记录」的索引以加速查找。

十一、对局智能分析

11.1 战力公式（v4 满配 + 三级降级）

每个比赛日对已结算对局计算「最强大腿 / 罪魁祸首 / 最佳拍档 / 最佳损友」四个榜单。战力指数以 KDA 为主，按数据维度是否齐全自动降级，保证任何数据条件下都能出结果。

档位	触发条件	公式
v4	含治疗+承伤+经济	(KDA×40% + 伤害×15% + 承伤×15% + 治疗×15% + 经济×15%) × 胜率
v3	无治疗，有承伤+经济	(KDA×40% + 伤害×25% + 承伤×20% + 经济×15%) × 胜率
v2	仅伤害+经济	(KDA×50% + 伤害×25% + 经济×25%) × 胜率
v1	仅 KDA	KDA × 胜率

其中 KDA =（击杀 + 0.4×助攻）/ max(死亡, 1)；伤害/经济/承伤/治疗均按「队内占比 × 队内人数」折算为倍率，再取该玩家所有局的平均值。战力最高的为「最强大腿」、最低的为「罪魁祸首」。

11.2 拍档与损友

对每局同队的所有两人组合统计「同队局数 / 同队胜场」，仅当同队局数 ≥ 2 才参评；胜率最高者为「最佳拍档」、最低者为「最佳损友」，同胜率时同队局数多者优先。

11.3 触发与定版

以下任一情况触发重算（regen_day）：录入/修改结果、撤销对局、LOL 导入或补全；此外，每晚 03:00 调度线程自动定版前一晚的分析。启动时 backfill_analytics() 会为「已结算对局涉及但尚无存档」的比赛日补档。

11.4 四舍五入一致性

后端 _half_up() 与前端 Math.round/toFixed 采用同样的半进位（round half up）规则，保证战力数值展示一致；前端渲染统一保留 1 位小数、胜率取整。

十二、前端逻辑（index.html）

12.1 页面结构

单页应用，三个标签页：抽签（选人 + 抽签分组展示 + 投票重抽）、玩家（添加玩家 + 积分排行榜 + 分模式切换）、战绩（智能分析 + 比赛日筛选 + 对战历史）。全局样式以白底 + 淡蓝主色、毛玻璃感卡片、紧凑布局为主；窄屏（<780px）单列、宽屏主容器加宽至 780px。

12.2 全局状态与同步

状态	说明
players / history	当前玩家与全部对局（来自 /api/state）
pendingSet	本局勾选玩家 id 集合
currentTeams	本端当前展示的分组 {A,B}
drawMode	抽签方式，强制 random（localStorage 记忆但会纠正为 random）
rankMode	积分榜模式：'all' 总榜 / 3 / 4 / 5（localStorage 记忆）
drawVotes / drawThreshold	当前投票数与达标阈值
drawSubmitting	本端分组提交中标志，防刷新覆盖刚抽的分组

refresh(force) 是统一同步入口：非 force 且距上次拉取不足 10 秒则直接返回（省流量）；否则拉 /api/state，更新 players/history、恢复上次勾选名单、渲染全部并同步服务器共享分组。切换标签页时调用 refresh()（走 10 秒节流），用户主动操作（抽签、录入、删除等）后调用 refresh(true) 强制刷新。

12.3 抽签页关键逻辑

1. renderPickGrid：按积分降序渲染玩家卡片，单击切换选中（选中态高亮 + 勾标）。
2. updatePickCount / updateDrawBtn：实时更新已选人数与「开始抽签」可用性；仅 6/8/10 人可抽；存在有效分组时禁用。
3. restoreLastPick：首次进入时把上次抽签名单恢复勾选（只恢复一次，之后用户手动改动不再覆盖）。

4. doDraw / startDrawFlow：预检服务器分组 → 播放洗牌动画（约 1.8 秒）→ 随机分组 → 展示 A/B 队卡片 → 提交服务器共享 → 启动 5 分钟倒计时。
5. renderSharedDraw：以服务器为唯一事实源渲染共享分组，含票数与倒计时；无分组时恢复选人区。

12.4 玩家页与排行榜

积分榜支持总榜（大数字 = 三模式积分总和）与 3v3/4v4/5v5 单榜（大数字 = 该模式积分）。每行右侧小字显示平均每局 KDA 比值 =（击杀 + 助攻）/ 死亡；零死亡且有击杀/助攻显示 Perfect，无数据显示「KDA —」。玩家名后可挂「本命英雄 / 抠脚英雄」标签。排序按当前榜积分降序，同分按昵称本地化排序。

12.5 本命 / 抠脚英雄算法

仅统计「有英雄名且带 KDA」的 LOL 对局，按玩家聚合各英雄的场次、胜场、KDA，加权综合分排序：场次 40% + 胜率 30% + KDA 30%。其中场次分 = 场次 / 该玩家最大场次 × 100（防止一两局的运气英雄被评为本命），胜率分 = 胜率 × 100，KDA 分 = min(KDA, 10) × 10（KDA ≥ 10 记满分，零死亡有击杀/助攻记满分）。综合分最高为本命、最低为抠脚。

12.6 战绩页

顶部为智能分析卡片（读 /api/analytics，跟随所选比赛日过滤），中部为比赛日筛选条（全部 / 各晚），下方为对战历史列表。每条历史显示局号、人数与分组方式、时间、两队成员（胜负底色区分），LOL 局额外显示 KDA、英雄名、伤害/承伤/治疗/经济（大数自动缩写为「万」）。

12.7 录入弹窗与只读约束

录入/修改弹窗（openRecordModal）支持选择胜方、标记四杀/五杀（仅列当前胜方队员，支持 ± 按钮多次标记）。当前版本已屏蔽抽签页「录入/修改本局胜负」与战绩页「修改结果 / 撤销 / 录入胜负」全部入口（对应 DOM 操作区 display:none），战绩仅由 LOL 自动同步写入，群成员只能查看不能修改；相关功能逻辑保留，可随时恢复显示。

十三、流量优化策略

花生壳免费版每月 1GB 流量，页面体积与刷新频率直接影响可用性。系统在服务端与前端两侧做了优化，将预计月流量从 3GB+ 降到约 300MB。

1. 服务端 gzip 压缩（_gzip_and_cache）：对 text/html、application/json、manifest、javascript、text/plain 等可压缩类型，在响应 ≥ 512 字节且客户端支持时压缩，约省 75% 流量；不压缩 PNG（已压缩格式）。
2. 浏览器缓存分级：图标与 manifest 缓存 7 天（重复访问零流量）；HTML 使用 no-cache（每次校验，命中 304 仅需几百字节）。
3. 前端刷新节流：refresh() 10 秒内的重复拉取直接复用本地数据，只有用户主动操作才用 force 强刷。
4. 首屏内容清空：pickGrid、rankList、analyticsBox、dayChips、historyList 等预渲染内容改由 JS 动态填充，减小 HTML 体积。

经优化后，index.html 由约 97KB 压缩到约 22KB，/api/state 由约 58KB 压缩到约 15KB。

十四、约束与运维要点

14.1 业务约束

- 抽签仅支持 6 / 8 / 10 人（3v3 / 4v4 / 5v5）；4v4 只能标四杀，5v5 四杀五杀均可标；3v3 无多杀加成。
- 当前分组 5 分钟失效；失效后需重新抽签。投票达半数解散分组、回到选人界面。
- 战绩只读：由 LOL 客户端自动同步，群成员不可手动录入/修改/撤销。
- 比赛日以 18:00 为界；凌晨 03:00 定版前一晚分析。

14.2 运行依赖

- 服务器电脑需开机并保持 Flask 服务运行（service_start.bat 窗口不关闭）；需要 Python 与 flask（脚本可自动安装）。
- 花生壳客户端需在线并登录（账号 cy19950726），映射内网 127.0.0.1:9529；固定域名已买断至 2099-12-31，永久有效。
- LOL 同步需 Windows、已登录打过自定义对局的账号、管理员权限；建议先选 [1] 预览核对名单再选 [2] 同步。

14.3 常见故障与处置

现象	处置
群页面打不开	确认服务器电脑已开机、Flask 服务在运行；双击桌面「启动抽签」图标可静默拉起并弹窗显示最新链接
LOL 同步报 RPC_ERROR / 503	客户端战绩服务未就绪：在客户端点开「生涯-战绩记录」触发加载，或完全退出重开客户端
同步提示未入玩家池	先在「玩家」页把该 Riot ID 加入玩家池（名字需能归一化匹配）
重复同步会不会重复计分	不会，服务器按 gameId 幂等去重，缺明细的局会自动补全
手机浏览器打开旧缓存	Ctrl+F5 强制刷新或清除站点缓存

14.4 变更纪律（重要经验）

> **曾发生一次 PowerShell 批量替换误将 index.html 清零的事故，最终从浏览器缓存中完整恢复。此后纪律：① 任何大改前必须备份（.temp 目录保留 index.pre-slim.bak、app.pre-slim.bak 等历史版本）；② 编辑大文件优先用脚本方式并配套校验；③ 修改后必须实测页面与接口，确认无功能缺失再交付。**

附录：完整文件清单

文件	说明
app.py	Flask 后端，全部业务逻辑（1031 行）
index.html	前端单页应用，含全部样式与脚本（1704 行）
elo_draw.db	SQLite 数据库文件
manifest.webmanifest	PWA 清单
icon-192.png / icon-512.png / apple-touch-icon.png	应用图标（双龙头 Logo）
lol_sync.bat / lol_sync.ps1	LOL 战绩同步工具
启动抽签.bat / 启动抽签.vbs / service_start.bat	一键启动链路
5v5均衡抽签.html	早期单文件版本（历史留存）
diagnose9.ps1 / diagnose10.ps1	LCU 连接诊断脚本
cpolar/	内网穿透备用工具目录
wegame/	相关工具目录

结语

本说明书覆盖了「对战盒子」从数据模型、计分规则、接口契约、算法细节到部署运维与前端交互的全部逻辑。后续如需扩展（如恢复均衡抽签、开放手动录入、增加模式或调整权重），可依据本文档定位对应代码位置：后端规则集中在 app.py，前端交互集中在 index.html。